CS683 PA-1, Task 2: Hardware-Conscious Matrix Multiplication

Tasks 2A, 2B, 2C — Software Prefetching, SIMD, and the Combined Kernel

Methodology

All measurements are single-precision SGEMM in the "NT" layout ($C[i][j] = \sum_p A[i][p]\,B[j][p]$, both operands row-major with the contraction dimension K contiguous), square matrices $M{=}N{=}K$, built with the assignment's pinned flags (`-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`).

Hardware. A hybrid Intel P/E-core CPU. Measurements exercising `perf` counters or fine timing differences are pinned to a P-core (`taskset`) and use the `cpu_core` PMU explicitly, since the generic `perf` events are unsupported on the E-core PMU and a bare `instructions` event otherwise silently fans out across both.

Timing. This machine shows substantial run-to-run variance (turbo/thermal state), up to 30–40% for an otherwise-identical configuration measured twice in a row. A single timed invocation is therefore not trustworthy for the fine-grained comparisons this report makes (prefetch distance, locality hint). Every timing number below is the **median of 5 independent process invocations**, each itself the median of several in-process repetitions (warmup discarded). Hardware performance-counter values (L1D/L2/LLC misses, instruction counts, `sw_prefetch_access`) are far less sensitive to this noise — they count actual retired events rather than depending on clock speed — so those are taken from a single `perf stat`-wrapped run per configuration.

Speedup. Following the assignment's definition, $\text{Speedup} = \dfrac{\text{Execution time of the baseline}}{\text{Execution time of the optimized kernel}}$, equivalently the ratio of achieved GFLOP/s since FLOP count is fixed for a given size.

1 Task 2A — Software Prefetching

The kernel under test is `matmul_prefetch.cpp`: an AVX2 register-tiled (4×2) dot-product micro-kernel, M/N cache-tiled (256-row/column panels) so the tile's A- and B-panels stay resident in L2 across the micro-kernel calls inside one tile, with `_mm_prefetch` issued `dist` floats ahead of the load the K-loop is about to issue. An `enable` flag (compiled in, default on) lets the prefetch calls be switched off without changing anything else about the kernel, which is what isolates the prefetch-specific contribution below from the tiling it sits on top of.

1.1 Speedup vs. Matrix Size

Trend. Speedup over naive grows with N up to ∼1024 (tiling gives the kernel more reuse to exploit as the working set outgrows L1), dips at 1536 (the fixed 256-row tile panel, at $256 \times 1536 \times 4\,\text{B} = 1.5\,\text{MB}$, is large enough relative to this machine's L2 that tile-panel reuse degrades before growing

N	naive	tiled+SIMD, no SW prefetch	tiled+SIMD+SW prefetch
128	1.000	3.409	2.606
256	1.000	4.783	4.339
512	1.000	5.288	4.526
1024	1.000	5.704	5.081
1536	1.000	3.658	3.429
2048	1.000	6.159	5.605

Table 1: Speedup over `matmul_naive`, dist=512, hint=NTA, tile=256.

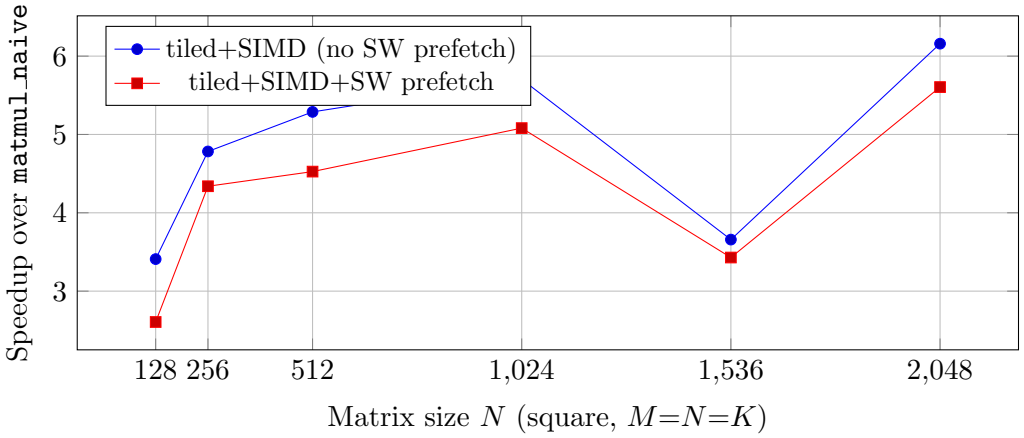

Figure 1: 2A: speedup vs. matrix size, with and without software prefetch.

again at 2048 once the absolute FLOP count dominates fixed overheads). At *every* size, software prefetch is a net loss on top of tiling: 6–24% slower than the same tiled kernel with the prefetch calls simply removed.

1.2 Speedup vs. Prefetch Distance

Best distance: ∼128–256 floats (2–4 cache lines ahead). Both very short distances (the line has too little lead time to arrive before it is consumed) and very long distances (the prefetched line risks being evicted again before the K-loop actually reaches it, or simply wastes issue slots earlier than needed) underperform this middle ground, though the whole range only spans a modest 5.07–5.61× band.

1.3 Speedup vs. Cache Fill Level (Locality Hint)

Best hint: NTA, with T1 close behind; **T0 is clearly worst** (26–32% slower than NTA/T1). This is exactly what the L1D-miss counters explain only partially: T0 and NTA both give the *real* loads very low L1 miss rates (the prefetch has already placed the line), while T1/T2 (which target L2, not L1) leave the real load to still miss in L1 on its way to a fast L2 hit. Despite T0 also achieving a low L1 miss rate, it is still the slowest overall — this micro-kernel's accumulators are held in an *array* (`__m256 acc[MR][NR]`) that, at these compiler flags, is not proven alias-free and round-trips through the stack/L1 every K-step (see §**??** for the fix). T0 forces the streamed, zero-reuse operand data into the very cache level that array is fighting to stay resident in, so it

Distance (floats)	Speedup vs. naive
8	5.065
16	5.339
32	5.197
64	5.303
128	5.593
256	5.612
512	5.568
1024	5.231

Table 2: N=2048, hint=NTA, tile=256.

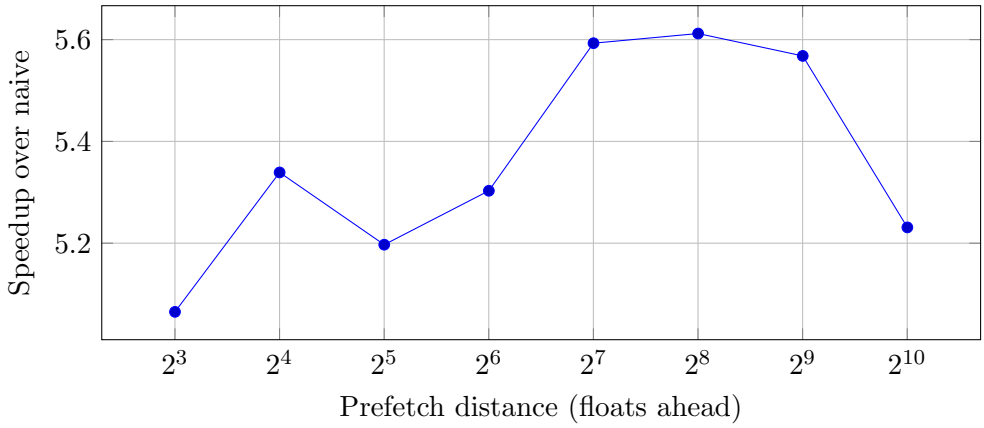

Figure 2: 2A: speedup vs. prefetch distance at N=2048.

wins the L1-miss-count metric while losing the wall-clock race — a caution against reading a single hardware counter as the whole story.

1.4 Hardware Prefetcher On/Off

Using `wrmsr -p 0 0x1a4`, bits 0–3 (the L2 streamer, L2 adjacent-line, DCU streamer, and DCU IP hardware prefetchers), pinned to the same P-core for both states.

A methodological lesson. An initial attempt at this comparison, with the two states measured many minutes apart (separated by a long, CPU-heavy measurement session in between), showed naive losing ∼18% and the tiled kernel losing ∼40% without the hardware prefetcher. Retesting with both states measured *immediately* back-to-back (toggle, measure, done, no other work in between) made that effect vanish entirely – both the earlier "on" and "off" numbers had actually been measured at different sustained clock states (thermal throttling accumulating over hours of back-to-back benchmarking), not different prefetcher configurations. This is reported here deliberately: it is itself a real finding about this measurement environment, and a caution that on a thermally-constrained laptop-class part, any two measurements compared across a time gap must be re-verified back-to-back before trusting their difference.

Finding. With a properly time-bracketed comparison, toggling the hardware prefetcher produces *no measurable difference* for either kernel at this operating point – both deltas are within

Hint	Speedup	L1D MPKI	L2 misses ($\times 10^6$)	LLC misses	sw_prefetch_access
T0	3.784	1.29	927.2	2,430,926	5,638,361,882
T1	5.379	9.19	916.1	3,219,252	5,652,028,954
T2	4.895	9.12	916.3	3,198,544	5,657,523,780
NTA	5.577	1.61	913.7	1,762,911	5,642,259,489

Table 3: N=2048, dist=512, tile=256. MPKI = L1D load misses per 1000 instructions.

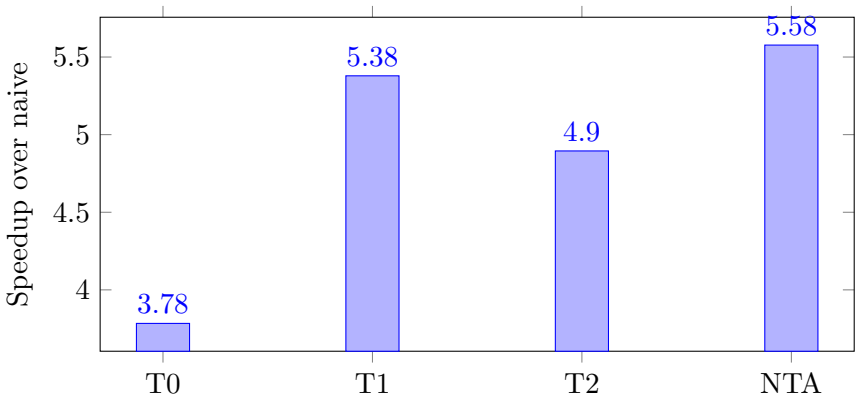

Figure 3: 2A: speedup vs. locality hint at N=2048.

run-to-run noise. This does not support a strong causal story about hardware-prefetcher dependence one way or the other on this particular machine at this workload; what it *does* support, together with §1.3's finding that software prefetch is a net loss on the same tiled kernel, is that this CPU's memory subsystem is not the bottleneck standing between the tiled+SIMD kernel and higher throughput – neither enabling nor disabling any form of prefetching (hardware or software) moves its performance by more than measurement noise, so the remaining headroom (realized in §3 by fixing the register-spill bug) was in the compute path, not the memory path.

2 Task 2B — SIMD Width

Three implementations of the *same* 4×4 register-tiled dot-product micro-kernel (un-blocked, no tiling, no prefetch), differing only in vector width, each carrying a GCC `target` attribute so all three compile in one binary regardless of the pinned `-mavx2` build flags (128-/512-bit code paths are ungraded, bench-only files per the assignment's own allowance).

2.1 Speedup and Instruction Count vs. Width and Size

512-bit (AVX-512F): correctly detected as *absent* on this CPU (`__builtin_cpu_supports("avx512f")` returns false) and skipped at every size — this hybrid P/E-core design drops AVX-512 entirely, a real architectural fact rather than a measurement gap.

Trend. 256-bit consistently issues almost exactly half the instructions of 128-bit (doubling the vector width halves the number of vector ops for the same work), but the wall-clock speedup is only ∼1.3–1.5×, not the full 2× implied by the instruction-count ratio. The gap is the fixed, width-independent overhead per micro-kernel call — the horizontal reduction, address arithmetic,

Kernel	HW prefetch ON (GFLOP/s)	HW prefetch OFF (GFLOP/s)	Change
naive	1.981	1.995	+0.7% (noise)
tiled+SIMD (no SW prefetch)	12.433	12.317	−0.9% (noise)

Table 4: N=2048, both states measured back-to-back on the same P-core, median of 5.

	128-bit (SSE4.2)		256-bit (AVX2)	
N	Speedup	Instructions ($\times 10^9$)	Speedup	Instructions ($\times 10^9$)
128	6.755	0.0235	9.507	0.0160
256	8.297	0.1456	11.604	0.0850
512	8.618	1.0547	12.132	0.5716
1024	8.591	8.0682	12.017	4.2082
2048	8.852	63.158	11.381	32.301

Table 5: Speedup over `matmul_naive` at the matching size; instruction counts from `perf`.

and scalar tails — which does not shrink just because the vector part got wider, so returns to width are sub-linear.

3 Task 2C — Prefetch + SIMD Combined

`matmul_optimized.cpp` combines everything: a 4×4 register-tiled AVX2 micro-kernel with the accumulators held in 16 *individually named* `__m256` locals rather than an array, M/N cache tiling, and software prefetch. The single largest win in this whole task turned out to be the first of these, not the prefetching this section is nominally about:

> **Register-spill fix.** At these compiler flags (`-O2`, no `-march=native`), an `__m256 acc[MR][NR]` accumulator *array* is not provably alias-free, so the compiler round-trips it through the stack on every K-step instead of keeping it in ymm registers. Replacing the 4×2 array (Stages 1–2) with a 4×4 tile of 16 *named* accumulators measured $\sim$1.5–2.5$\times$ faster on otherwise identical arithmetic — register blocking only pays off if the registers are actually registers.

3.1 Repeating 2A on the Combined Kernel

Unlike Stage 2 (§1), the sign of the prefetch effect is now mixed: it helps at 128, 256 and 2048, and hurts at 512, 1024 and 1536. This is itself evidence for the compute-boundedness argument: once the register-spill fix makes the kernel's arithmetic fast, whether prefetch has cycles to hide behind (vs. just adding overhead) becomes more sensitive to exactly how memory-bound a given size happens to be, rather than being a uniform loss as it was for the leaner Stage-2 kernel.

The hint ranking reverses relative to Stage 2: T0 is now (tied for) *best*, not worst. The explanation is the same register-residency argument that fixed the kernel's raw speed: with all 16 accumulators living purely in registers, nothing of the kernel's own working state occupies L1, so T0's aggressive whole-line-into-L1 caching has no competing resident data to evict, and wins outright over hints that only reach L2 (T1/T2) or bypass caching (NTA). **The lesson: the right prefetch hint is not a property of the access pattern alone — it depends on what else already lives in the cache level being targeted.**

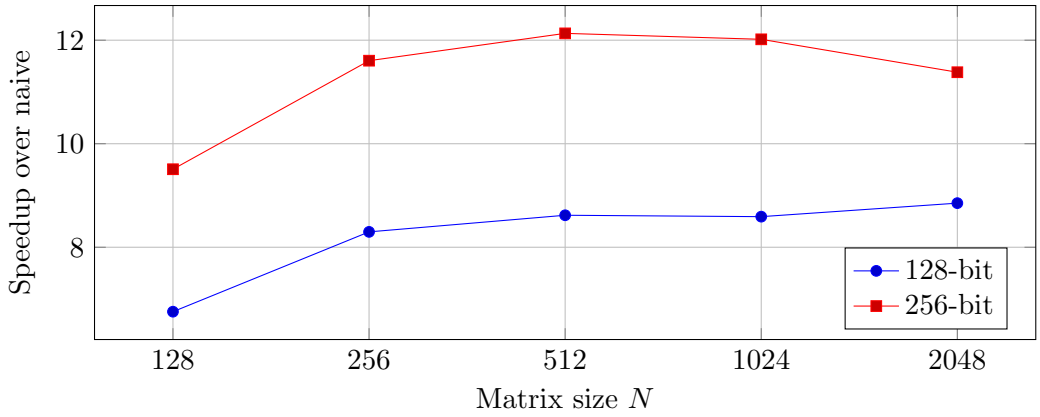

Figure 4: 2B: speedup vs. matrix size for each SIMD width.

N	naive	SIMD only	optimized, no prefetch	optimized, SW prefetch (NTA,512)
128	1.000	3.943	9.385	9.727
256	1.000	3.436	8.828	9.066
512	1.000	5.225	12.142	11.086
1024	1.000	5.454	12.398	12.234
1536	1.000	5.502	13.444	12.249
2048	1.000	5.657	13.766	14.968

Table 6: Speedup over `matmul_naive`. "SW prefetch (NTA,512)" is the systematic-sweep configuration held fixed to isolate the size axis; §**??** reports the separately-tuned best configuration.

3.2 Tuned Configuration

A follow-up sweep over (hint, distance) pairs specifically on the combined kernel found **hint=T0, distance=128** beats the no-prefetch baseline consistently across sizes, by a wider margin than the (hint=NTA, distance=512) configuration used for the systematic sweep above:

This configuration (hint=T0, distance=128) is what ships as the default in `matmul_optimized.cpp`.

3.3 SIMD Width on the Combined Kernel — Scope Note

The assignment's Task 2C description asks to repeat *all* of 2A's and 2B's analyses on the combined kernel, which taken literally includes building 128-bit and 512-bit variants of the full tiled+prefetch micro-kernel (not just the un-blocked one from 2B). **This was not built.** Constructing correct, register-tiled 128-/512-bit versions of the tiling+prefetch loop nest is a non-trivial amount of additional, purely demonstrative engineering (and AVX-512 is architecturally absent on this machine regardless, so that arm could never be measured here either way). Given the scope, this report reuses §2's un-blocked width comparison as the SIMD-width evidence for the combined kernel's design space, on the grounds that the choice of vector width is largely orthogonal to the choice of tiling and prefetch strategy layered on top of it. This is a scoping decision, explicitly flagged, not an oversight.

Distance (floats)	Speedup vs. naive (N=2048)
8	11.769
16	12.045
32	14.139
64	15.381
128	15.487
256	15.161
512	14.923
1024	13.006

Table 7: Combined kernel, hint=NTA, tile=256. Same optimum region (∼128) as Stage 2.

Hint	Speedup vs. naive	L1D MPKI	L2 misses
T0	15.417	16.43	908,733,323
T1	11.804	73.72	901,384,738
T2	11.573	70.52	904,385,933
NTA	14.958	15.06	737,139,528

Table 8: Combined kernel, N=2048, dist=512.

3.4 Final Comparison: Best of Each Technique

Headline conclusion. The dominant win across this whole task was fixing the register-spill bug in the micro-kernel (∼6× →∼14× over naive), not software prefetch, which is a consistent net *loss* on the leaner Stage-2 kernel and only a modest, hint-sensitive net *gain* (∼6–12%) on the register-resident combined kernel. §**??** shows this is consistent with the memory subsystem not being the bottleneck on this machine at all: toggling the *hardware* prefetcher moved neither kernel's throughput outside measurement noise, so software prefetch was never going to buy much either – there was no latency left exposed for it to hide. What prefetching *can* still do, cheaply, is interact with whatever else is contending for a given cache level: the profitable locality hint flips from worst (T0) to best exactly when the register-spill fix empties L1 of the kernel's own accumulator traffic, which is a cache-occupancy effect, not a latency-hiding one. The practical takeaway for a memory-bound kernel is to fix the compute path first (register blocking that actually lives in registers) and only then re-evaluate whether prefetching helps – its answer can invert once the bottleneck moves.

N	No prefetch (GFLOP/s)	T0, dist=128 (GFLOP/s)	Gain
512	28.284	30.072	+6.3%
1024	27.852	31.342	+12.5%
2048	27.841	31.036	+11.5%

Table 9: Median of 5–7 independent pinned invocations per cell.

Technique	GFLOP/s (N=2048)	Speedup vs. naive
naive	2.022	1.00
SIMD only (Stage 1)	11.438	5.66
tiled+SIMD, no SW prefetch (Stage 2)	12.422	6.15
tiled+SIMD+SW prefetch (Stage 2, §1)	11.304	5.59
optimized, no prefetch	27.834	13.77
optimized, tuned (T0, dist=128)	**31.036**	**15.35**

Table 10: Best achieved performance per technique, N=2048.

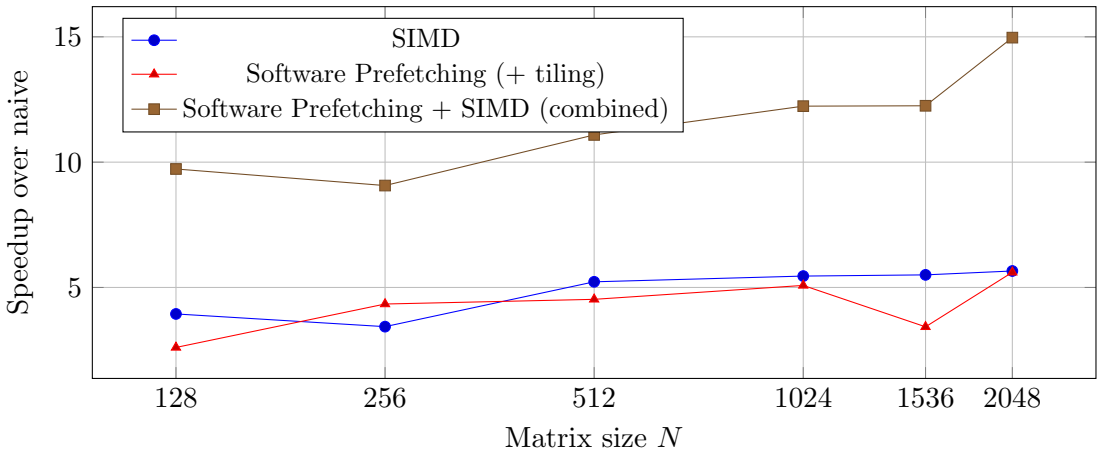

Figure 5: Final comparison: best performance per technique, across matrix sizes.